ReportaTuCiudad

Fase 2: Planteamiento del proyecto | Entrega: 15 de octubre de 2026

Equipo: Luis Guillermo Hernandez Casillas, Carlos Emiliano Olmedo Navarro, Isaian Ayala García | Materia: Desarrollo en la Nube | Repositorio: https://github.com/luishdezc/ReportaTuCiudad.git | Video: https://youtu.be/Yvr5PMDE6xI

1. La aplicación y el problema que resuelve

ReportaTuCiudad es una plataforma web que conecta a los ciudadanos con una cuenta oficial municipal simulada para reportar, dar seguimiento y cerrar problemas urbanos: baches, luminarias dañadas, acumulación de basura, fugas de agua, árboles caídos o semáforos fuera de servicio. El ciudadano crea una cuenta, levanta un reporte con descripción, fotografía y ubicación, y lo ve publicado en un mapa junto con los reportes de su zona; además puede confirmar que observó un problema ya existente y comentar en él. La cuenta municipal recibe los reportes en un panel administrativo, cambia su estado (Reportado, En revisión, Asignado, En proceso y Resuelto), responde y publica la evidencia de la solución, momento en el que el ciudadano recibe una notificación y confirma si el problema realmente quedó atendido.

El problema que atacamos es la falta de trazabilidad en el reporte de incidencias urbanas. Hoy los canales típicos (teléfono, redes sociales o ventanilla) no dan un folio verificable, no evitan reportes duplicados del mismo bache y no avisan al ciudadano cuando su reporte avanza, por lo que la gente deja de reportar y la autoridad no sabe qué problemas afectan a más personas. La plataforma vuelve el proceso visible de extremo a extremo y usa las confirmaciones de la comunidad como señal de prioridad.

2. Por qué una aplicación en la nube es la solución adecuada

La carga de trabajo es intermitente e impredecible: puede haber pocos reportes durante días y cientos tras una lluvia fuerte o un socavón en una avenida principal. Un esquema serverless en AWS escala automáticamente en esos picos y cuesta prácticamente cero cuando nadie reporta, algo imposible de lograr con un servidor propio encendido todo el semestre. El proyecto también maneja archivos pesados: cada reporte lleva fotografías y evidencias, y S3 los almacena de forma duradera y barata sin saturar la base de datos. Además, el servicio debe estar disponible desde cualquier celular en la calle y publicar información en un mapa compartido, lo que exige un backend accesible por internet y con alta disponibilidad. Finalmente, la nube nos permite desacoplar el envío de notificaciones con colas y mensajería, aplicar seguridad por roles y monitorear el sistema con métricas y alarmas sin instalar ni mantener infraestructura, de modo que el equipo dedica el semestre a construir la aplicación y no a administrar servidores.

3. Flujos end to end que se van a desarrollar

- **Flujo 1. Levantar un reporte ciudadano:** el ciudadano inicia sesión, llena el formulario (categoría, descripción y ubicación en el mapa) y adjunta una foto que se sube a S3 con una URL prefirmada; API Gateway invoca la Lambda de reportes, que valida los datos, guarda el reporte en DynamoDB con estado Reportado y publica un evento en SQS. El reporte queda visible en el mapa público y el ciudadano recibe su folio.
- **Flujo 2. Gestión y cambio de estado por la cuenta municipal:** la cuenta oficial entra al panel administrativo, filtra los reportes por zona, categoría o número de confirmaciones, abre uno y cambia su estado (En revisión, Asignado, En proceso, Resuelto), escribe una respuesta y, al cerrarlo, sube la evidencia de la solución. Cada cambio se guarda como historial en DynamoDB y dispara el evento que SQS y SNS convierten en la notificación por correo al ciudadano que reportó y a quienes confirmaron el problema.
- **Flujo 3. Participación comunitaria y confirmación del cierre:** cualquier usuario consulta el mapa, abre un reporte existente, lo confirma con "yo también lo vi" (para evitar duplicados y sumar prioridad) y comenta en él. Cuando el reporte se marca como Resuelto, quienes participaron reciben la notificación con la evidencia y responden si el problema quedó solucionado; si la mayoría indica que no, el reporte se reabre.

4. Servicios de AWS que se van a usar

Servicio	Uso en ReportaTuCiudad	Por qué se eligió
Amazon API Gateway	Expone la API REST del prototipo (/reportes, /comentarios, /admin) y valida los tokens de sesión.	Da un único punto de entrada seguro (HTTPS), controla el tráfico y se integra de forma directa con Lambda sin administrar servidores web.
AWS Lambda	Ejecuta la lógica de negocio: alta y consulta de reportes, confirmaciones, comentarios y cambio de estado desde el panel municipal.	Modelo serverless de pago por ejecución: escala solo ante picos de reportes y evita administrar servidores, lo que se ajusta al presupuesto y al tiempo de un proyecto escolar.
Amazon DynamoDB	Almacena usuarios, reportes, comentarios, confirmaciones e historial de estados.	Base NoSQL administrada con latencia baja y consultas por índice (zona, categoría, estado); su esquema flexible se adapta a los distintos tipos de reporte.
Amazon S3	Guarda las fotografías del ciudadano y las evidencias de solución publicadas por la cuenta municipal, mediante URLs prefirmadas.	Almacenamiento de objetos económico y duradero para archivos pesados, que además aloja el sitio estático del frontend.
Amazon SQS + Amazon SNS	SQS encola los eventos de nuevo reporte y cambio de estado; SNS envía la notificación por correo al ciudadano y a quienes siguen el reporte.	Desacoplan la notificación de la petición del usuario: la respuesta de la API es inmediata y ningún aviso se pierde si falla un envío.

Servicios de soporte: AWS IAM (roles con permisos mínimos entre servicios), AWS Secrets Manager (llaves y credenciales de la aplicación) y Amazon CloudWatch (logs, métricas, alarmas y un dashboard del prototipo).

5. Infraestructura como código: Terraform

Todos los recursos del prototipo se crearán con Terraform y no a mano desde la consola de AWS. Un solo conjunto de archivos .tf, versionado en el repositorio, describirá las tablas de DynamoDB, los buckets de S3, las funciones Lambda, la API de API Gateway, la cola de SQS, el tópico de SNS, los roles de IAM y las alarmas de CloudWatch. Esto nos permite reconstruir el ambiente completo con un solo comando, mantener ambientes separados de desarrollo y demostración con las mismas definiciones, revisar cambios de infraestructura por pull request y eliminar los recursos con terraform destroy cuando no se usen para no consumir los créditos de la cuenta. El estado de Terraform se guardará en un bucket de S3 con bloqueo en DynamoDB para que los tres integrantes puedan aplicar cambios sin sobrescribirse.

6. Roles del equipo

Integrante y rol	Responsabilidades principales
[Carlos Olmedo] Líder de proyecto, arquitectura cloud y monitoreo	Coordina al equipo y el cronograma, define la arquitectura en AWS, escribe los módulos de Terraform, administra la cuenta y los roles de IAM, configura SQS, SNS y Secrets Manager, y arma el dashboard y las alarmas de CloudWatch.
[Luis Hernandez] Backend, API y datos	Implementa las funciones Lambda y los endpoints de API Gateway, el modelo de datos en DynamoDB, la carga de imágenes y evidencias a S3 con URLs prefirmadas, y la lógica de cambio de estados.
[Isaian Ayala] Frontend, pruebas y documentación	Desarrolla la interfaz del ciudadano y el panel municipal (formulario de reporte, mapa, comentarios y seguimiento), ejecuta las pruebas de los tres flujos y mantiene el repositorio de GitHub y los entregables.

7. Boceto del diagrama de arquitectura

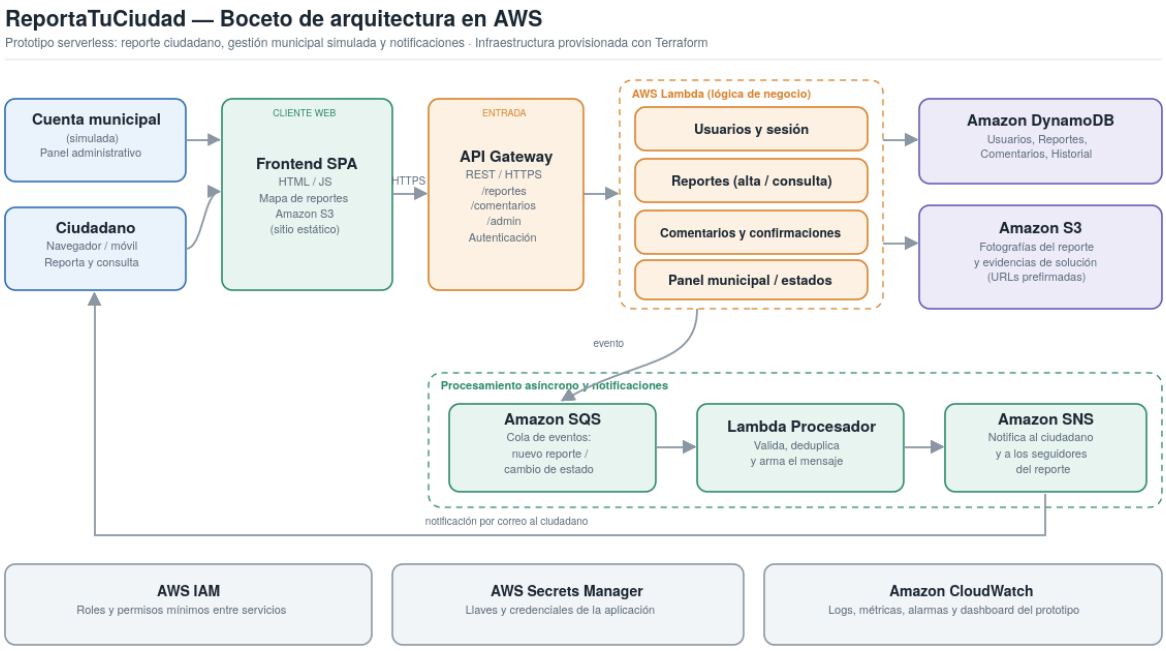


Figura 1. Arquitectura serverless del prototipo. Versión interactiva en el repositorio: /docs/arquitectura.html

8. Visión del entregable final

Al terminar el semestre entregaremos un prototipo funcional desplegado en AWS y accesible desde una URL pública, con los tres flujos operando de punta a punta sobre datos de prueba. La aplicación tendrá registro e inicio de sesión, alta de reportes con foto y ubicación, mapa público con filtros, confirmaciones y comentarios, panel de la cuenta municipal simulada con cambio de estados y evidencia de solución, y notificaciones automáticas por correo en cada avance.

- Repositorio de GitHub con el código del frontend y del backend, los módulos de Terraform que levantan la infraestructura y el README del proyecto.
- Demostración en vivo del ciclo completo: un ciudadano reporta un bache, la comunidad lo confirma, la cuenta municipal lo atiende y el ciudadano valida el cierre.
- Dashboard de CloudWatch con métricas del prototipo (reportes creados, errores, latencia de la API y mensajes en cola) y al menos una alarma configurada.
- Documentación final: diagrama de arquitectura actualizado, modelo de datos, contrato de la API y un manual breve de despliegue con Terraform.

Actualización. Fase 2

Esta sección completa el reporte con el estado del proyecto al cierre de la Fase 2.

9. Enlaces del proyecto

- **Demostración en video:** [pegar aquí el enlace al video]

10. Descripción actualizada y estado del proyecto

La aplicación y el problema que resuelve son los de la sección 1 y no cambiaron; lo que cambió es que dejó de ser una propuesta. El prototipo está desplegado y funcionando en AWS, con los tres flujos completos, y toda su infraestructura se levanta con trece archivos de Terraform y un solo comando. El inventario de componentes está en la sección 14.

11. Flujos que se van a demostrar

- **Flujo 1. Levantar un reporte ciudadano.** Registro, formulario con categoría, descripción y ubicación en el mapa, y foto que el navegador sube directo a S3 con una URL prefirmada. Queda en DynamoDB como Reportado, aparece en el mapa con su folio y dispara la notificación.
- **Flujo 2. Gestión y cierre por la cuenta municipal.** Bandeja ordenada por confirmaciones, cambio de estado con respuesta e historial. Cerrar en Resuelto exige evidencia, y cada cambio notifica al autor y a quienes confirmaron.
- **Flujo 3. Participación comunitaria y validación del cierre.** Confirmaciones “yo también lo vi” aceptadas una sola vez por usuario, comentarios, y validación final del ciudadano; si dos o más responden que el problema sigue, el reporte se reabre.

12. Servicios de AWS: qué se conserva y qué cambió

Se conservan los cinco servicios de la sección 4, más IAM, Secrets Manager y CloudWatch. Al implementarlos surgieron tres cambios:

Servicio	Cambio respecto a la Fase 1	Motivo
API Gateway	HTTP API en lugar de REST API	Menor costo y latencia, y CORS nativo; no necesitamos lo avanzado de REST API
Amazon S3	Dos buckets: fotografías y evidencias, y el sitio estático	Separa lo que suben los usuarios del código del frontend, con políticas distintas
AWS IAM	En la cuenta de clase las Lambdas reutilizan el rol LabRole	El Learner Lab no permite crear roles; en una cuenta normal Terraform crea los suyos

13. Decisiones técnicas y su justificación

- **Sesiones con JWT propio firmado con HMAC-SHA256, en vez de Amazon Cognito.** La llave vive en Secrets Manager y las contraseñas como hash PBKDF2. Evita dependencias en el paquete Lambda y deja el flujo de autenticación visible para la materia.
- **Tres tablas en DynamoDB en lugar de una sola.** Medimos ambos diseños con 2,000 reportes sintéticos: la diferencia es de medio RRU por consulta, y las tres tablas dan permisos de IAM por entidad y un esquema que se lee sin traducir prefijos.
- **SQS antes de SNS, y notificación por tópico del equipo.** La API responde sin esperar al correo y la DLQ evita perder avisos. Suscribir a cada ciudadano exigiría Amazon SES, así que el correo incluye a quiénes se notificaría.
- **Los buckets de S3 los crea el script de despliegue, no el recurso de Terraform.** Ese recurso consulta la configuración de Object Lock al leerse y una política de la cuenta de clase niega esa llamada. Terraform administra toda la demás configuración.

Pendientes conocidos: la confirmación escribe el registro y el contador por separado y el cambio de estado no valida el estado previo, así que ambos deben pasar a transacciones; y el mapa sin filtro usa un Scan que conviene sustituir por una consulta al índice.

14. Componentes en la nube y componentes locales

- **En la nube (AWS), alrededor del 90 % del proyecto:** sitio web y panel municipal (S3) · API de trece rutas (API Gateway) · cinco funciones (Lambda) · usuarios, reportes e interacciones (DynamoDB) · fotos y evidencias (S3) · eventos y notificaciones (SQS, DLQ y SNS) · llave de firma (Secrets Manager) · permisos (IAM) · logs, alarmas y dashboard (CloudWatch).
- **En la máquina del equipo:** Terraform y la CLI de AWS, los scripts de despliegue, de siembra de la cuenta municipal y de prueba de los flujos, y el repositorio local. No son parte de la aplicación: son herramientas que operan sobre la nube.
- **Servicios de terceros:** Leaflet y los mosaicos de OpenStreetMap, que el navegador carga desde una CDN pública. Es lo único del producto final que no se sirve desde AWS.